

PMSCS 622P

Programming Languages

Lecture 05: Function and Recursion

Mr. Md. Masum Bhuiyan

Department of Computer Science and Engineering
Jahangirnagar University

Lecture Roadmap

- **Function Syntax and Mechanics**
- **Recursion Fundamentals**
- **Complexity Analysis of Recursion**
- **Optimization of Recursion**

1.1 What is a Function?

Formal Definition

A **function** (also called a method, procedure, or subroutine) is a named sequence of statements that performs a specific computation.

1.2 The "Black Box" Analogy

Think of a function as a machine in a factory.



Example: The `sqrt(x)` function.

- Input: 25
- Internal: Complex Math (Newton-Raphson method?) → *We don't care!*
- Output: 5

2.1 Anatomy of a Function Declaration

A function needs three things:

- 1 **Name:** To call it later.
- 2 **Parameters:** The variables it accepts (The Inputs).
- 3 **Body:** The code that runs.

The Return Type:

- **C++/Java:** You must declare what type of data comes out (e.g., int, void).
- **Python:** The language figures it out dynamically.

2.2 Comparative Syntax: Defining a Function

C++

```
// Type Name (Params)  
int add(int a, int b) {  
    return a + b;  
}
```

Requires types. Uses braces {}.

Java

```
// Modifiers Type Name  
public static int add(int  
    a, int b) {  
    return a + b;  
}
```

Must be inside a Class. `static` means "no object needed".

Python

```
# def Name (Params):  
def add(a, b):  
    return a + b
```

Uses `def`. No braces, uses **indentation**.

2.3 Invoking (Calling) a Function

Defining a function does nothing. You must **call** it to run the code.

```
// Logic Flow
int main() {
    int x = 10;
    int y = 20;

    // 1. Program Pauses here
    // 2. Jumps to 'add' function
    // 3. 'result' gets the value 30
    int result = add(x, y);

    // 4. Program resumes
}
```

2.4 Parameters vs Arguments

These terms are strictly defined in Computer Science:

Parameter (The Variable)

The placeholder defined in the function signature.

```
void func(int x) ...  
"I expect an integer here."
```

Argument (The Data)

The actual value passed when calling the function.

```
func(5);  
"Here is the integer 5."
```


2.5 Void vs Fruitful Functions

Fruitful Functions: Return a value. Can be assigned to a variable.

- `y = sin(x)`

Void Functions: Perform an action but return nothing.

- `print("Hello")` or `saveToFile(data)`

C++ / Java

```
void greet() {  
    cout << "Hi";  
    // No return statement  
}
```

Python

```
def greet():  
    print("Hi")  
    # Returns 'None' implicitly
```

3.1 Variable Scope

Scope determines where a variable is visible.

- 1 **Local Scope:** Variables defined inside a function. They are created when the function starts and **destroyed** when it ends.
- 2 **Global Scope:** Variables defined outside all functions. Visible everywhere (but dangerous).

3.2 Local Variable Lifetime

```
void myFunction() {  
    int secret = 99; // Local to myFunction  
}  
  
int main() {  
    myFunction();  
    // ERROR: 'secret' does not exist here.  
    // It was deleted from memory when myFunction finished.  
    cout << secret;  
}
```

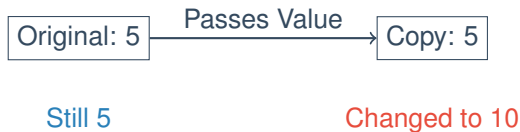
Metaphor: Local variables are like hotel guests. Once they check out (function ends), the room is cleaned, and no trace remains.

3.3 Passing Data: Pass by Value

The Default in C++ and Java (Primitives).

When you pass a variable to a function, the function gets a **photocopy** of the value.

- If the function changes the photocopy, the original is **unaffected**.



3.4 Passing Data: Pass by Reference

Sometimes we *want* the function to change our variable. We pass the **Memory Address**.
C++ (Using '&')

```
void upgrade(int &score) {  
    score = 100; // Modifies original  
}  
  
int main() {  
    int s = 50;  
    upgrade(s);  
    // s is now 100  
}
```

Python (Mutable Objects)

```
def upgrade(my_list):  
    my_list.append(100)  
  
data = []  
upgrade(data)  
# data is now [100]
```

4.1 Recursion: The Concept

Recursion is a method of solving a problem where the solution depends on solutions to smaller instances of the same problem.

In Programming

A function that calls itself.

Why use it?

- Some problems are naturally recursive (Trees, Graphs, File Systems).
- Code is often cleaner and more mathematical than loops.

4.2 The Three Laws of Recursion

To write a recursive function, you must satisfy three conditions:

- 1 **Base Case:** A condition that stops the recursion. Without this, you get an infinite loop.
- 2 **Recursive Call:** The function must invoke itself.
- 3 **Reduction Step:** The arguments must move toward the base case (e.g., n becomes $n - 1$).

4.3 Case Study: Factorial

Mathematically, Factorial ($n!$) is defined as:

$$n! = n \times (n - 1) \times (n - 2) \dots \times 1$$

This can be rewritten recursively:

$$n! = n \times (n - 1)!$$

Base Case: $0! = 1$ or $1! = 1$.

4.4 Factorial Implementation

C++ / Java

```
int fact(int n) {  
    // 1. Base Case  
    if (n == 0)  
        return 1;  
  
    // 2. Recursive Call & Reduction  
    return n * fact(n - 1);  
}
```

Python

```
def fact(n):  
    # 1. Base Case  
    if n == 0:  
        return 1  
  
    # 2. Recursive Call & Reduction  
    return n * fact(n - 1)
```

4.5 Tracing Execution: The Descent

Let's trace `fact(3)`.

- **Step 1:** Call `fact(3)`. Check `3 == 0`? No. Return `3 × fact(2)`.
- **Step 2:** Call `fact(2)`. Check `2 == 0`? No. Return `2 × fact(1)`.
- **Step 3:** Call `fact(1)`. Check `1 == 0`? No. Return `1 × fact(0)`.
- **Step 4:** Call `fact(0)`. Check `0 == 0`? **YES**. Return 1.

Notice: No multiplication has happened yet! We just piled up requests.

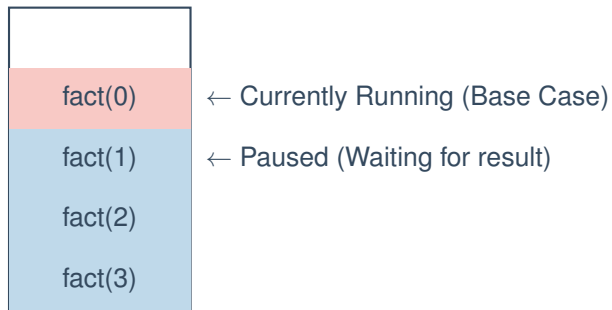
4.6 Tracing Execution: The Ascent

Now the returns bubble up:

- **Step 4 returned 1.**
- **Step 3:** Receives 1. Calculates 1×1 . Returns **1**.
- **Step 2:** Receives 1. Calculates 2×1 . Returns **2**.
- **Step 1:** Receives 2. Calculates 3×2 . Returns **6**.

Final Result: 6.

4.7 The Call Stack



Each function call adds a "Frame" to the stack. Frames consume memory.

4.8 The Recursion Trap: Stack Overflow

What happens if we forget the Base Case?

- The function calls itself infinitely.
- New frames are added to the stack forever.
- Computer memory (RAM) allocated for the stack runs out.
- **Crash!**

Java: `java.lang.StackOverflowError`

Python: `RecursionError: maximum recursion depth exceeded`

5.1 Linear vs Branching Recursion

Linear Recursion (e.g., Factorial):

- The function makes **1** recursive call.
- Shape: A straight line.
- Complexity: $O(N)$.

Branching Recursion (e.g., Fibonacci):

- The function makes **2 or more** recursive calls.
- Shape: A tree.
- Complexity: Often Exponential ($O(2^N)$).

5.2 Fibonacci Sequence

The sequence: 0, 1, 1, 2, 3, 5, 8, 13...

$$F(n) = F(n - 1) + F(n - 2)$$

$$F(0) = 0, F(1) = 1$$

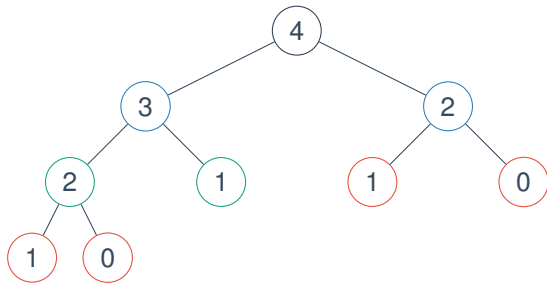
To calculate $F(5)$, we need $F(4)$ and $F(3)$. To get $F(4)$, we need $F(3)$ and $F(2)$...

5.3 Fibonacci Implementation

```
int fib(int n) {  
    if (n <= 1) return n; // Base Cases  
  
    // Double Recursive Call  
    return fib(n-1) + fib(n-2);  
}
```

This looks simple, but it is dangerous for large N .

5.4 Visualizing the Tree (n=4)



5.5 Time Complexity Analysis

To estimate time, we count the total nodes in the tree.

- Depth 0: 1 node
- Depth 1: 2 nodes
- Depth 2: 4 nodes
- Depth k : 2^k nodes

Total Nodes $\approx 2^n$.

Time Complexity: $O(2^n)$.

This is Exponential Time. Calculating Fib(50) would take years on a standard laptop.

5.6 Space Complexity

Space depends on the **Stack Height**.

- Even though the tree is wide, the computer explores one branch at a time (Depth First Search).
- The maximum number of frames on the stack at once is N .

Space Complexity: $O(N)$.

6.1 The Problem: Overlapping Subproblems

Look back at the Fibonacci Tree.

- We calculated **Fib(2)** on the left branch.
- We calculated **Fib(2)** *again* on the right branch.

In larger trees, we recalculate the exact same values millions of times. This is inefficient.

6.2 The Solution: Memoization

Idea: When we solve a problem (like $\text{Fib}(2)$), write the answer down in a table. Before solving a problem, check if the answer is already in the table.

Dynamic Programming Preview

This technique transforms $O(2^n)$ algorithms into $O(n)$ algorithms. (*We will cover this in Lecture 9: Data Structures/Algorithms*).

6.3 Optimization: Tail Recursion

If the recursive call is the **last action** in the function, smart compilers (C++) can turn the recursion into a loop automatically.

```
// Tail Recursive
void printDown(int n) {
    if (n == 0) return;
    cout << n << endl;
    printDown(n - 1); // Last action
}
```

Note: Java and Python generally do NOT support Tail Call Optimization (TCO).

Practice 1: Sum of Digits

Problem: Write a recursive function that takes an integer (e.g., 1234) and returns the sum of its digits ($1 + 2 + 3 + 4 = 10$).

Hint:

- $1234 \% 10 = 4$ (Last digit)
- $1234 / 10 = 123$ (Remaining digits)

Solution 1: Sum of Digits

Algorithm:

- 1 Base Case: If $n < 10$, return n .
- 2 Recursive: Return (last digit) + sum(remaining).

```
def sum_digits(n):  
    # Base Case  
    if n < 10:  
        return n  
  
    # Recursion  
    return (n % 10) + sum_digits(n // 10)
```


Practice 2: Power Function

Problem: Write a recursive function `power(base, exp)` that calculates $base^{exp}$.

Example: `power(2, 3)` -> 8.

Logic:

$$2^3 = 2 \times 2^2$$

$$2^0 = 1$$

Solution 2: Power Function

```
int power(int base, int exp) {  
    // Base Case  
    if (exp == 0) return 1;  
  
    // Recursive Step  
    return base * power(base, exp - 1);  
}
```

Trace power(2,3):

- $2 \times \text{power}(2, 2)$
- $2 \times (2 \times \text{power}(2, 1))$
- $2 \times (2 \times (2 \times \text{power}(2, 0)))$
- $2 \times 2 \times 2 \times 1 = 8$

Practice 3: Palindrome Check

Problem: Check if a string is a palindrome (reads same forwards and backwards).
Example: "radar", "madam".

Recursive Strategy:

- 1 Compare first and last characters.
- 2 If they match, strip them off and recurse on the middle substring.
- 3 If string length is 0 or 1, it is a palindrome.

Solution 3: Palindrome

```
def is_palindrome(s):  
    # Base Cases  
    if len(s) <= 1:  
        return True  
  
    # Check bounds  
    first = s[0]  
    last = s[-1]  
  
    if first == last:  
        # Slice string: remove first and last  
        middle = s[1:-1]  
        return is_palindrome(middle)  
    else:  
        return False
```

Practice 4: Reverse a String

Problem: Write a recursive function to reverse a string.
Input: "Hello" → Output: "olleH"

Hint:

- $\text{reverse}(\text{"Hello"}) = \text{reverse}(\text{"ello"}) + \text{"H"}$

Solution 4: Reverse String

```
public static String reverse(String str) {  
    // Base Case: Empty or single char  
    if ((str == null) || (str.length() <= 1)) {  
        return str;  
    }  
  
    // Recursive Step  
    return reverse(str.substring(1)) + str.charAt(0);  
}
```

Summary

- 1 **Functions** are the building blocks of clean code.
- 2 **Syntax:** Python is concise; C++/Java are strict/verbose.
- 3 **Recursion** relies on the Call Stack. It needs a Base Case.
- 4 **Complexity:**
 - Single Recursion: $O(N)$
 - Double Recursion: $O(2^N)$ (Avoid without memoization!)

Common Errors to Watch For

1. Infinite Recursion

Forgetting the base case or not reducing the problem size. Result: Stack Overflow.

2. Off-by-one Errors

Base case logic is slightly wrong (e.g., stopping at 1 instead of 0).

3. Global Variable Abuse

Trying to use global variables to track recursion state often leads to bugs. Pass state as arguments instead.